



J. J. Strossmayer University of Osijek

Faculty of Electrical Engineering,
Computing and Information
Technologies Osijek
Kneza Trpimira 2b
HR-31000 Osijek
www.ferit.unios.hr

Lab Assignment 2:
**Time Synchronization in Distributed Computing
Systems**

Academic year 2025/2026.

Zdravko Krpić, Ph.D, Associate Professor

When a 37-Second Bug Stopped the World

In June 2012, web services across the globe suddenly crashed at once. Airlines couldn't check passengers in, websites went offline, and even some data centers froze. The culprit wasn't hackers or hardware failures, but a small leap second.

Coordinated Universal Time (UTC) occasionally adds an extra second to keep clocks in sync with Earth's rotation. But many Linux systems at the time weren't prepared. Their kernels entered a tight loop when that leap second occurred, consuming 100% CPU and locking up entire servers.

This single timing error disrupted large-scale services like Reddit, LinkedIn, and Qantas Airways' booking system. The incident showed how even a tiny discrepancy in time synchronization can cascade into massive real-world failures.

Since then, organizations have developed more robust strategies for handling leap seconds, including "leap smear" techniques that spread the extra second across the day.

Exercise 2: Time Synchronization in Distributed Computing Systems

1 Introduction

1.1 Why Time Synchronization Matters

In a distributed system, each machine has its own hardware clock. These clocks are never perfectly aligned: they drift apart due to small hardware differences and are affected by network latency when exchanging messages.

Without synchronization, we cannot rely on timestamps across nodes. This can lead to problems such as:

- Misordered event logs (an effect may appear to happen before its cause).
- Inconsistent database updates in replicated systems.
- Security errors (e.g., authentication or certificate checks failing because of mismatched time).
- Failures in scheduling or coordination between processes.

Real-world example:

Google's *Spanner* database uses both atomic clocks and GPS receivers in its data centers to keep time synchronized with high precision, because even a small drift can cause inconsistencies in a globally distributed system.

1.2 Two Approaches to Synchronization

There are two main approaches to dealing with time in distributed systems:

1.2.1 Physical Clocks

The goal is to make each machine's clock match a reference such as UTC.

- **Cristian's Algorithm:** a client asks a time server for the correct time and adjusts its clock.
- **Berkeley Algorithm:** a master node polls all nodes, computes an average time, and instructs nodes to adjust.
- **NTP (Network Time Protocol):** the real-world system used today, based on layered time servers.

Limitations: due to network delays, clocks can never be perfectly synchronized.

1.2.2 Logical Clocks

Instead of using "real" time, we maintain a logical ordering of events.

- **Lamport Clocks:** each event gets a monotonically increasing counter.
- **Vector Clocks:** extend Lamport clocks to capture causality more precisely.

Logical clocks do not tell "when" something happened in real-world time, but they guarantee that if one event happened before another, the timestamps will reflect that order.

1.3 A Simple Example of the Problem

Imagine two nodes, A and B, with unsynchronized clocks:

- Node A's local time = 100
- Node B's local time = 90

Now:

1. Node A sends a message at its local time = 100.
2. Node B receives the message at its local time = 91.

If we look at the logs:

B: Received message at t=91 A: Sent message at t=100

It looks like the message was received before it was sent! This paradox makes it impossible to reason about event ordering unless we synchronize clocks.

1.4 What You Will Do Today

In this lab, you will:

1. Simulate a distributed system with nodes and local clocks.
2. Implement **Lamport logical clocks** to guarantee consistent event ordering.
3. Extend the system with more advanced synchronization algorithms (optional, time permitting).

By the end, you will understand both the limitations of physical clock synchronization and the power of logical clocks for maintaining order in distributed systems.

We will start with a guided example that shows how unsynchronized clocks lead to problems, and then you will implement Lamport clocks to fix it.

2 Guided Code Example: Time Synchronization with MPI

In this part of the lab, we will explore how unsynchronized clocks in a distributed system can lead to paradoxical results, and how Lamport logical clocks can restore consistency.

Each MPI process represents a *node* with its own **local clock**. To simulate clock drift, we intentionally add an **offset** (positive or negative) to each rank's local clock. Rank 0 acts as the **sender** and Rank 1 as the **receiver**.

We will run the program in two modes:

1. **Unsynced mode** – nodes rely only on their local (offset) wall-clock time.
 - Problem: the receiver may record a timestamp **earlier** than the sender's.
2. **Lamport mode** – nodes maintain and exchange **Lamport clocks**.
 - Fix: Lamport timestamps guarantee a consistent ordering of events, even when wall clocks disagree.

2.1 How to Compile and Run

Compile the program (use either mpicc for C or mpic++ for C++ depending on the provided source):

```
mpicc -std=c11 -O2 mpi_time_demo.c -o mpi_time_demo
```

or

```
mpic++ -std=c++17 -O2 mpi_time_demo.cpp -o mpi_time_demo
```

Run with two processes:

```
mpirun -n 2 ./mpi_time_demo <mode> <offset_rank0_seconds> <offset_rank1_seconds>
```

- <mode> is either unsynced or lamport
- <offset_rank0_seconds> and <offset_rank1_seconds> are integers or floats that simulate the local clock offset for each process.

Examples

```
mpirun -n 2 ./mpi_time_demo unsynced 5 -4
mpirun -n 2 ./mpi_time_demo lamport 5 -4
```

2.2 What to Observe

- In **unsynced mode**:
 - Output may show `recv_wall < sender_wall`.
 - This is the **paradox**: Node B seems to receive a message before Node A sends it.
- In **lamport mode**:
 - Each send/receive updates the Lamport clock.
 - The log shows `L_in` and `L_after` values that guarantee `L_after > L_in`.
 - Even if wall-clock times are inconsistent, the **Lamport clocks establish a valid partial order**.

2.3 Source code

Be sure to read the inline comments, which highlight where the paradox occurs and how Lamport's algorithm solves it.

```

#include <mpi.h>
#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <iostream>
#include <thread>
#include <algorithm>

// Simple helpers
double local_time_with_offset(double offset_seconds) {
    // MPI_Wtime() is per-process wall clock; we inject an artificial offset.
    return MPI_Wtime() + offset_seconds;
}

void sleep_ms(int ms) {
    std::this_thread::sleep_for(std::chrono::milliseconds(ms));
}

// -----
// UNSYNCED DEMO (paradox)
// -----

// -----
// LAMPORT DEMO (causal order)
// -----

// -----
// MAIN
// -----
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank = 0;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (argc < 4) {
        if (rank == 0) {
            std::cerr << "Usage: " << argv[0]
                << " <unsynced|lamport> <offset_rank0_sec> <offset_rank1_sec>\n";
        }
        MPI_Finalize();
        return 1;
    }

    std::string mode = argv[1];
    double off0 = std::atof(argv[2]);
    double off1 = std::atof(argv[3]);

    if (mode == "unsynced") {
        run_unsynced(off0, off1);
    } else if (mode == "lamport") {
        run_lamport(off0, off1);
    } else {
        if (rank == 0) {
            std::cerr << "Unknown mode: " << mode << " (use 'unsynced' or 'lamport')\n";
        }
        MPI_Finalize();
        return 2;
    }

    MPI_Finalize();
    return 0;
}

```

Figure 1. Common infrastructure code.

```

// -----
// UNSYNCD DEMO (paradox)
// -----
void run_unsynced(double off_rank0, double off_rank1) {
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (size < 2) {
        if (rank == 0) std::cerr << "Need -n 2 or more.\n";
        return;
    }

    const double my_off = (rank == 0) ? off_rank0 : off_rank1;
    MPI_Barrier(MPI_COMM_WORLD); // start together

    if (rank == 0) {
        // Sender: send 3 messages to rank 1; include sender wall-time
        for (int seq = 1; seq <= 3; ++seq) {
            double t_send = local_time_with_offset(my_off);
            // Send payload: [seq(int), t_send(double)]
            int header[1] = { seq };
            MPI_Send(header, 1, MPI_INT, 1, 100, MPI_COMM_WORLD);
            MPI_Send(&t_send, 1, MPI_DOUBLE, 1, 101, MPI_COMM_WORLD);
            std::printf("[A:rank0] SENT seq=%d wall=%.6f\n", seq, t_send);
            std::fflush(stdout);
            sleep_ms(300);
        }
        // End signal
        int end = -1; MPI_Send(&end, 1, MPI_INT, 1, 100, MPI_COMM_WORLD);
    } else if (rank == 1) {
        // Receiver: log local (offset) wall-time vs sender's wall-time
        while (true) {
            int seq;
            MPI_Status st1{}, st2{};
            MPI_Recv(&seq, 1, MPI_INT, 0, 100, MPI_COMM_WORLD, &st1);
            if (seq == -1) break; // end
            double sender_wall;
            MPI_Recv(&sender_wall, 1, MPI_DOUBLE, 0, 101, MPI_COMM_WORLD, &st2);

            double t_rcv = local_time_with_offset(my_off);
            std::printf("[B:rank1] RECV seq=%d rcv_wall=%.6f sender_wall=%.6f\n",
                        seq, t_rcv, sender_wall);
            std::fflush(stdout);
        }
    }
}

```

Figure 2. Paradox Demonstration Function (to be combined with Figure 1).

```

// -----
// LAMPORT DEMO (causal order)
// -----
void run_lamport(double off_rank0, double off_rank1) {
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (size < 2) {
        if (rank == 0) std::cerr << "Need -n 2 or more.\n";
        return;
    }

    const double my_off = (rank == 0) ? off_rank0 : off_rank1;
    int L = 0; // Lamport clock

    auto on_send = [&]() -> int {
        return ++L;
    };
    auto on_recv = [&](int L_msg) -> int {
        L = std::max(L, L_msg) + 1;
        return L;
    };

    MPI_Barrier(MPI_COMM_WORLD); // start together

    if (rank == 0) {
        // Sender: send 3 messages with Lamport & wall-time
        for (int seq = 1; seq <= 3; ++seq) {
            int L_out = on_send();
            double t_send = local_time_with_offset(my_off);

            // Payload parts:
            // header_ints: [seq, L_out]
            int header_ints[2] = { seq, L_out };
            MPI_Send(header_ints, 2, MPI_INT, 1, 200, MPI_COMM_WORLD);
            MPI_Send(&t_send, 1, MPI_DOUBLE, 1, 201, MPI_COMM_WORLD);

            std::printf("[A:rank0] SENT seq=%d L=%d wall=%.6f\n", seq, L_out, t_send);
            std::fflush(stdout);
            sleep_ms(300);
        }
        // End
        int end_header[2] = {-1, -1};
        MPI_Send(end_header, 2, MPI_INT, 1, 200, MPI_COMM_WORLD);
    } else if (rank == 1) {
        // Receiver: update Lamport on receive and log both times
        while (true) {
            int header_ints[2];
            MPI_Status st1{}, st2{};
            MPI_Recv(header_ints, 2, MPI_INT, 0, 200, MPI_COMM_WORLD, &st1);
            int seq = header_ints[0];
            int L_in = header_ints[1];
            if (seq == -1) break;

            double sender_wall;
            MPI_Recv(&sender_wall, 1, MPI_DOUBLE, 0, 201, MPI_COMM_WORLD, &st2);

            double t_recv = local_time_with_offset(my_off);
            int L_after = on_recv(L_in);

            std::printf("[B:rank1] RECV seq=%d L_in=%d L_after=%d recv_wall=%.6f sender_wall=%.6f\n",
                seq, L_in, L_after, t_recv, sender_wall);
            std::fflush(stdout);
        }
    }
}

```

Figure 3. Lamport Causal Order Function (to be combined with Figure 1).

3 Student Activity: Implementing Synchronization

In this part of the lab, you will apply what you observed in the guided example.

You will implement **Lamport logical clocks** yourself, starting from a simplified MPI boilerplate program.

3.1 Objectives

- Implement Lamport clock updates on **send** and **receive**.
- Attach Lamport timestamps to messages.
- Verify that the resulting logs respect causal order, even when wall-clocks disagree.
- (Optional) Extend your solution with **Vector Clocks** or **Cristian's algorithm** for physical synchronization.

3.2 Provided Boilerplate Code

Below is a simplified MPI program with two ranks (0 = sender, 1 = receiver).

The **MPI setup and message passing are already provided**. Your task is to complete the parts marked with `//`
`TODO`.

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <algorithm>

// Global Lamport clock
int L = 0;

// TODO: Implement Lamport update rules
int on_send() {
    // increment clock before sending
    // return the Lamport value to attach to message
    return 0; // placeholder
}

int on_receive(int L_msg) {
    // update Lamport clock with max rule
    // return the new Lamport value
    return 0; // placeholder
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        // Sender: send 3 messages
        for (int seq = 1; seq <= 3; seq++) {
            int L_out = on_send(); // TODO: implement
            int payload[2] = { seq, L_out };
            MPI_Send(payload, 2, MPI_INT, 1, 0, MPI_COMM_WORLD);
            printf("[A] Sent seq=%d L=%d\n", seq, L_out);
        }
        int end[2] = { -1, -1 };
        MPI_Send(end, 2, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
    else if (rank == 1) {
        // Receiver: log Lamport updates
        while (1) {
            int payload[2];
            MPI_Status status;
            MPI_Recv(payload, 2, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);

            if (payload[0] == -1) break; // end signal
            int seq = payload[0];
            int L_in = payload[1];
            int L_after = on_receive(L_in); // TODO: implement
            printf("[B] Received seq=%d L_in=%d L_after=%d\n", seq, L_in, L_after);
        }
    }

    MPI_Finalize();
    return 0;
}

```

Figure 4. Boilerplate for Lamport Clock Implementation.

4 Assignments

Assignment 1. Lamport Clock Implementation (mandatory):

- Complete `on_send()` and `on_receive()` to update the global Lamport clock correctly.
- On send: increment before sending.
- On receive: $L = \max(L, L_{\text{msg}}) + 1$.

Assignment 2. Verification:

- Run the program with different clock offsets (simulate with `sleep()` or by adding delays).
- Check that while wall-clock times may contradict, Lamport timestamps always increase consistently.

Assignment 3. Optional Extensions (for faster groups):

- **Vector Clocks:** replace the integer Lamport clock with an array $V[]$, one entry per process. Implement update rules.
- **Cristian's Algorithm:** modify the code so rank 0 acts as a "time server" and rank 1 adjusts its local clock. Measure the offset error.

Example Run (expected structure)

```
[A] Sent seq=1 L=1
[B] Received seq=1 L_in=1 L_after=2
[A] Sent seq=2 L=3
[B] Received seq=2 L_in=3 L_after=4
...
```